Koord: Language and simulator for distributed cyberphysical systems

ABSTRACT

We present the design and implementation of a new programming language for distributed cyberphysical systems (CPS) called *Koord* that separates the platform-independent computations from platform-specific concerns of sensing, communication, and low-level control. *Koord* raises the level of abstraction for programming CPS by providing distributed shared memory for coordination, it allows parameterized controllers for portability across different physical platforms, and it is based on a familiar event-triggered concurrent programming model. We present the formal semantics of *Koord* in the $\mathbb{K}$ executable semantic framework. This narrows the gap between the theoretical model of *Koord* and its implementation. We present an implementation of *Koord* that generates executable Python code, which has been deployed on robotic platforms. Finally, we introduce a high-fidelity simulator and show how it can be used to test, visualize, and debug large deployments of *Koord* applications. We illustrate the various features and the design decisions involved through applications such as platooning, formation flight, and distributed task allocation.

KEYWORDS

mobile robotics, swarms, simulation, semantics

ACM Reference Format:

. 2018. Koord: Language and simulator for distributed cyberphysical systems. In ,. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/1122445.1122456

1 INTRODUCTION

Motivation. Programming applications that interact with physical processes are becoming central in robotics, manufacturing, IoT, and other cyberphysical systems (CPS). Following the trends in the cloud and smartphone ecosystems, programmability is going to become important in these domains as platform become more open and hardware developers shift to the applications marketplace. Unfortunately, *domain specific languages (DSL)* for these domains are surgically platform dependent, and they combine low-level sensing, communication, and control tasks with the application-level logic [**?**]. This tight-coupling of applications with platform-specific details can hinder development, modularity, portability, code reuse, and testing.

This need for raising the level of abstraction and separating the *platform-independent* from the *platform-dependent* concerns motivates our work in this paper. Consider for a moment, the task of developing an application for distributed package delivery with mobile robots in a building. The higher-level coordination has to deal with assigning robots to visit way-points in different rooms, load-balancing, and mainteinence schedules. For all of the reasons mentioned above, these coordination tasks *should be* separated from the steering control of the individual vehicles, indoor positioning, and message-level communication protocols tied to the specific hardware platforms.

Current programming languages do not natively support such separation of concerns. They do not have the high-level abstractions needed for programming distributed cyberphysical systems. The developer will have to keep track of communication between different sensors, actuators, and program variables. The *robotic operating system (ROS)* [**? ?**] does provide useful libraries and publish-subscribe messaging standards, but these are not integrated in a programming abstraction. Similarly, coordination across multiple robots has to be built from the ground up using message passing threads. In order to simulate and test the application, one has to create different threads or instances for the developed code corresponding to different agents, and on top of that, the agent programs have to be harnessed in a physical world simulator with proper sensor and actuator models. In order to analyze correctness, one has to first define the semantics of the system—a difficult proposition in an of itself, given the concurrency and data flows in multiple time-scales.

Our project aims to address some of these challenges by designing, developing, and evaluating an open source programming system for distributed robotics. [1] In this paper, we present for the first time the design and the implementation of the *Koord language* and the supporting software tools, namely the compiler, the $\mathbb{K}$ executable semantics, and the simulator. These components form the core of the programming system. *Koord* programs have been deployed on F1/10 vehicles, quadcopter, and roombas, and the results related to hardware deployment will be presented in a future article.

1.1 Contributions of the paper

1.1.1 Design and implementation of Koord language. We present a clean-slate design and implementation of an event-triggered programming language, for distributed cyberphysical systems, namely Koord. *Koord* combines distributed shared memory abstraction for coordination across agents and a synchronous model of communication with the physical environment through sensors and actuators. Individual agent programs, which may be concurrent, are written in a familiar precondition-effect style language.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

,

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9999-9/18/06…$15.00
https://doi.org/10.1145/1122445.1122456

[1] Links and references to the project as well as downloadable software will be made available to the PC chairs. We omit the information here for the sake of maintaining anonymity.

```
1  using Motion:
2    sensors pos psn
3    actuators target
4
5  allread: pos x
6
7  TargetUpdate:
8    pre True
9    eff if not(pid == N_sys − 1 or pid == 0):
10     Motion.target = mid(<x[pid+1],x[pid−1]>)
11     x[pid] = Motion.psn
```

Figure 1: Lineform *Koord* program

Consider a simple line-formation program in which a set of N_{sys} robots form a equi-spaced line starting from arbitrary positions. The textbook algorithm is for each robot i to repeatedly move towards the midpoint of the line joining the positions of $i-1$ and $i+1$. The extremal robots with ids 0 and $N_{sys}-1$ stay fixed. This is a archetypal protocol for synchronization, pattern formation, and consensus [**? ? ? ?**] in distributed robotics. The above 10 line *Koord* program implements this line formation algorithm and can be simulated or deployed (see Figure 2). Among these 10 lines of code, lines 1-5 import the *controller module* called *Motion* that enables this program to access the sensor port with the agent's position (*psn*) and the actuator port (*target*). An implementation of a *controller module* has platform-specific path planners and low-level controllers for moving specific types of robots in the physical world. *Thus, with different implementations of the* Motion *interface the Lineform Koord program can be ported to different platforms.*

Another powerful feature of *Koord* used in the *Lineform* program is the single-write multi-reader (**allread**) shared array x, in which component $x[i]$ records the position of the ith robot in each round. Shared variables allow the participating agents programs to coordinate without exposing the programmer to explicit message passing channels, buffers, and threads. This makes *Koord* programs succinct, readable, and often, as in the above example, remarkably close to the textbook version of the algorithm. The *Koord* runtime system (discussed in more detail in Section 4) implements the propagation of the shared variable write over messages. The formalization of the resulting semantics is discussed in Section 3.

The one and only event in the *Lineform* program is *TargetUpdate* (lines 5-9): it updates the target position of the agent to be the midpoint of its neighbors. Figure 2 shows the result of simulating a slightly modified version of *Lineform* on the *Koord* simulator with 25 robots forming a 3D-shape. The *Koord* compiler can also generate executables that can be deployed on several mobile robotic hardware platforms such as F1/10 cars [**?**], drones, and Roombas.

1.1.2 First formalization of a robotics or CPS language in $\mathbb{K}$ *.* Implementations of programming languages usually contain many bugs. One common type of bugs creeps in from the gap between the official semantics of the language (as in a user manual) and the its implementation as embodied in a compiler. The $\mathbb{K}$ framework [**?**] closes this gap by allowing the specification of *formal executable semantics of any programming language* in terms of rewrite rules. These rewrite rules define how each and every possible statement in the language changes, possibly nondeterministically, the "state of the machine" executing the program. $\mathbb{K}$ provided the most complete to-date formalization of the C language [**?**]. This semantics was tested against the GCC torture test suite and it successfully passed 99.2% of 776 test programs. Recently, $\mathbb{K}$ was used to provide a formal semantics of x86-64 [**?**] and the Ethereum Virtual Machine (EVM) bytecode.

In order to close the above-mentioned gap, we have developed the complete executable formal semantics of Koord in $\mathbb{K}$ *.* This is the first formalization of a robotics or a CPS or a multi-agent language in $\mathbb{K}$. One technical challenge here is to define the "state of the machine" executing *Koord* programs. This state now has to include not only the states of the program variables of the different agents, but also the sensor and actuator ports reflecting the state of the physical world, and the state of shared variables as implemented over messages.

Our key innovation for achieving this formalization is to parameterize the semantics with a set of *environment functions* that model the behavior of the physical environment, the controller, and other operations that are external to the application program. The environment functions are called by the back-end of $\mathbb{K}$ to update certain state components (e.g., sensor and actuator ports). Thus, the formal semantics of *Koord* is also modular and portable across different environment models.

Koord programs have a discrete event loop, which executes periodically to interact with the environment. We can view the semantics of *Koord* as a nondeterministic, timed automaton [**?**]: there is a *sampling parameter* $\delta > 0$; every δ-time units *at most one event* of the agent's program executes. This execution takes zero logical time and corresponds to a transition of the timed automaton. *Koord* provides distributed shared variable abstraction for programming multiple agents, which can be written to during an event execution. The consistency semantics implemented here is that the writes to shared variables are propagated to all the agents, and become visible to other agents reading the variable after one round (δ time units). In the fault-free synchronous model considered here, any number of gossip-based algorithms can be used to implement this semantics in the *Koord* runtime system.

Our $\mathbb{K}$ implementation of *Koord* enables us to execute programs *semantically accurately*. One of the factors resulting in nondeterminism is the order in which agents execute their events, and we can use the $\mathbb{K}$ support for nondeterminism to generate and search all possible nondeterministic behaviors in bounded executions for a given program. To further support formal analysis in case of non-specific initial states, we also implemented an interval arithmetic in $\mathbb{K}$, and support execution of *Koord* programs where the initial values of variables are provided in intervals. We implemented a *Koord bounded-model-checking (*KoordBMC*)* tool on top of the $\mathbb{K}$ semantics. While the state space explosion due to the nondeterminism and distributed nature of the CPS applications makes it difficult for *KoordBMC* to scale to beyond a handful of agents, the tool is useful for finding bugs in smaller instances. For instance, in a platooning application written in *Koord* (Section 5.1), *KoordBMC*

can be used to determine the value of the sampling parameters for which the safety invariant is violated.

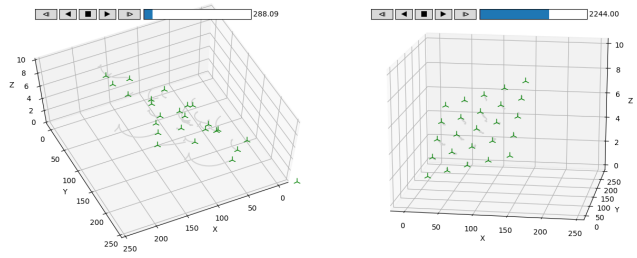

Figure 2: Screenshot of *Koord* simulator visualizing positions of a swarm of 25 agents running a slightly modified version of *Lineform* application forming a 3D-shape in space.

1.1.3 Koord simulator and applications. We have developed a high-fidelity, multi-threaded simulator for Koord applications. The simulator can execute instances with 50+ agents with heterogeneous dynamics, executing *Koord* applications, and therefore, it is a powerful tool for debugging and performance analysis.

The major challenges involved in designing a simulator engine with multiple threads for each agent for managing event loops, messaging, and sensor-actuator updates. The simulator uses a *motion automaton*, which can be instantiated to simulate different robot dynamics, sensors, etc. The communication protocols to propagate shared memory messages between agents are also a part of the simulator communication module. The time between two computational steps (or discrete event loop iterations) is used to propagate messages to implement shared memory as in the actual hardware stack. The versatile nature of the *Koord* and the simulator are illustrated further with three examples—platooning, swarm formations, and distributed task allocation—in Section 5.

The rest of the paper is organized as follows: In Section 2, we resent a brief discussion of related research. In Section 3, we present the detailed semantics of *Koord* including the executable semantics in $\mathbb{K}$. In section Section 4, we give an overview of the supporting software we have developed including the runtime system for executing *Koord* programs and *KoordBMC*. We present the applications in Section 5 and conclude in Section 5.4.

2 RELATED WORK

Modern programming languages like C# and Swift, and compiler infrastructures like LLVM [?] have revolutionized the application development ecosystem in mobile computing. Inspired by these successes, there is a surge of interest in open and portable languages that raise the level of abstraction [? ? ? ?] (For an earlier survey of Domain Specific Programming Languages (DSLs) for robotic systems see [?]. Most of these older languages are proprietary or generate executable files that are tied to specific platforms). Buzz [?] and React [?] fall in this category as does our language *Koord*. The Live Robot Programming language [?] not only provides a higher-level programming abstraction in terms of nested state machines, but also allows the program to be changed while running, hence reducing the feedback loop across writing, compiling, and testing of robot programs. Buzz currently does not connect with verification tools, and the verification approach implemented with React uses precise models of the environment and performs model checking using dReal [?]. "Correct-by-construction" synthesis from high-level temporal logic specifications has been applied to mobile robotic systems (see, for example [? ? ? ? ?]). Our point of view on automating robot programming is different in that we expect that the programmer's creativity and efforts will be necessary well beyond writing high-level specs in solving distributed robotics problems; consequently only the tedious and standard steps in coordination and control are automated using the *Koord* compiler.

Programming systems using the shared memory paradigm have been developed for several distributed computing systems [? ? ? ? ?]. Specifically, P [?] and PSync [?] are DSLs for asynchronous partially distributed systems, but cyberphysical interactions are not supported.

3 KOORD LANGUAGE AND SEMANTICS

The interaction model of an agent executing a *Koord* program is shown in Figure 3. The program and the controller interact with

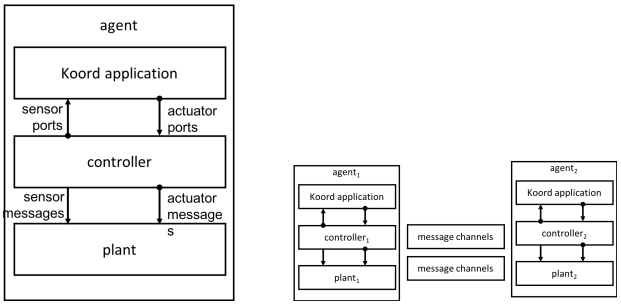

Figure 3: An agent executing a *Koord* application and interacting with the physical environment through sensor and actuator ports (*left*). Interaction across multiple agents happens through shared variables implmented over message passing (*right*).

each other through *sensor* and *actuator* ports. The program reads the sensor ports, reads and writes to program variables, and writes to actuator ports. The controller drives the underlying physical plant. The *environment* of a *Koord* program is the combination of the controller and the plant. The set of agents in the system may interact with each other using *shared variables*. Each agent in the system runs an instance of the *same Koord* program.

A distributed Task allocation application. Figure 4 shows a distributed task allocation application written in *Koord*. This app involves mutual exclusion as well as construction of de-conflicted paths. Tasks are abstractions for real location-based objectives like package delivery, surveillance, or fire-fighting. Koord enables implementation of simple task allocation strategies using shared variables. In *Task* app, the agents maintain a shared list of tasks (*taskList*) to complete, and each agent is uniquely assigned a task from this list in the event *Assign*. Once an agent reaches the task location (event *Reached*), it does whatever the task entails (event *Complete*), and continues this process.

```
1  Task(pid)
2
3  def task :
4    pos loc
5    int asgnid
6    int taskId
7
8  using Motion
9    sensors:
10     pos psn,bool done
11   actuators:
12     List<pos> route
13
14 allwrite:
15   list<task> taskList
16
17 allread:
18   List<pos> routes[pid] = Motion.psn
19
20 local:
21   int i = 0
22   path p
23   task temp, new
24   enum Stage {ASGN,RCHD,CMPT} stage
25                = ASGN
26
27 Assign:
28   pre stage == ASGN
29   eff atomic:
30     for (i = 0,i < length(taskList),i = i+1):
31       temp = taskList(i)
32       p = getPath(temp.loc)
33       if temp.asgnid == 0 and notNull(p,routes):
34         new = new Task(temp.loc,pid,temp.taskId)
35         put(taskList,i,new)
36         routes[pid] = p
37         Motion.route = p
38         stage = RCHD
39
40 Reached:
41   pre stage = RCHD and Motion.done
42   eff routes[pid] = temp.loc
43       stage = CMPLT
44
45 Complete:
46   pre stage == CMPLT
47   eff stage = ASGN
```

Figure 4: *Koord* **program for robot** i **to do required tasks at provided locations.**

3.1 Koord model for distributed CPS

Koord design comprises of a *turn*-based alternating semantics of discrete and dynamic behavior of each agent in the distributed system. The agents can be viewed as performing *rounds* of execution, where each round consists of a (1) *a program transition* which is a discrete computational step, and (2) (3) *an environment transition* which is the dynamic behavior of the controller, environment, and their effect on the sensors and actuators . In our semantics, we record whether it is the turn for a program transition or an environment transition by a special *turn* variable.

In a program transition in a round, each agent first nondeterministically chooses an *enabled* event, or an event whose precondition evaluates to `True`. It then executes the statements in the effect of that event. Then, the agent performs the environment transition in the round, during which it interacts with the physical environment and behaves as dictated by the actuator ports on the controller for δ amount of time. [2]

The controller which determines the dynamic behavior of each agent and its interaction with the environment. The agent program can read from the sensor ports read and write to the actuator ports during the program transition. The sensor ports are updated during the environment transition according to the controller dynamics.

Compared to δ, the time taken by the agent to execute the computational step is negligible, and in our formalization we treat it as zero logical time. Because each program transition takes zero logical time, we can assume all the agents start and finish executing their program transitions at the same logical time. As our design choices impose this synchronicity on all agent executions, the system can also be seen to be executing program and environment transitions by *turns*, where all the agents execute the environment transitions in a round only after all of them have finished executing the program transitions in that round (in some order).

$$\underbrace{C_1 \rightarrow_{stmt} C_2 \rightarrow_{stmt} \ldots \rightarrow_{stmt} C_n}_{\text{program transition}} \rightarrow_{env} C'_n$$

We proceed to discuss the the formal semantics of the *Koord*. The BNF grammar for the language is given in the appendix.

3.2 Program (cyber) variables and physical variables

We first introduce the types of variables used by the *Koord* programs.

Program variables. An agent's *Koord* program can access three types of variables.

- *Local program variables* record the state of the program. For example, the variable p of type *path* (Line 22) stores a path for the agent.
- *Distributed shared variables* are used for communication across agents. For example, **allwrite** *taskList* (Line 14) is a multi-writer list of tasks which can be written-to and read-from by all participating agents. Shared variables can also be single-writer multi-reader (**allread**). For each shared variable, an agent maintains a *local copy* of the variable.
- *Port variables* are used to read from and write to sensor and actuator ports of the agent. For example, the *Motion* controller for the *Task* app drives the agent through a route, as directed by value set at the actuator port *Motion.route*. The sensor port *Motion.psn* gives the position of the agent (in a fixed coordinate system) and *Motion.done* indicates whether the *Motion* controller is active or inactive.

The valuation of these variables define the configuration for an agent. The agent program also has access to two system-level constant parameters (a) a unique integer identifier *pid* for itself and (b) a list *ID* of identifiers of all participating agents[3].

Agent configurations. The syntax and grammar for *Koord* is given in the Appendix. Let us fix P to be a syntactically correct agent program. Let *Var* be the set of program and shared variables in P. Let *Sens* be the set of sensor ports of the controller used in P. Let *Val* be the set possible valuations that these variables can take. A *configuration* of agent i, $i \in ID$, running *Koord* program P is a tuple $L_i = (pc, M, sm, turn)$, where

- pc is a syntactic production that generates P.
- $M : Var \mapsto Val$ is a *local context* mapping each variable to a corresponding valuation.
- $sm : Sens \mapsto Val$ is a mapping of each sensor port to a valuation.
- $turn : \{\texttt{prog}, \texttt{env}\}$ is a bookkeeping variable; $turn = \texttt{env}$ indicates that agent i has finished executing its program for the current round.

$\mathbb{L}$ denotes the set of all possible agent configurations. The components of an agent configuration L_i are accessed using the usual dot

[2] $\delta > 0$ is a constant *sampling period* parameter. The sampling parameter can be provided by the user, and each controller that *Koord* supports has a default sampling parameter which has been set based on the experiments using the controller. See Section 5 for more details.

[3] Our current system implementation assumes that the set of possible participating agents is known. This assumption will be relaxed in the future.

$(\cdot)$ notation. That is, $L_i.M$ refers to the local context M in agent configuration $L_i \in \mathbb{L}$, for the agent with pid i in the system. Consider the Task application in Figure 4. The local value of the variable p in line22 for agent with *pid* 0 is represented as $L_0.M(p)$.

3.3 System configurations

We define the semantics of the overall system in terms of a nondeterministic, discrete-time transition system or a system automaton. The state of the automaton is defined in terms of *system configurations* consisting of the agent configurations and the valuation of the shared variables.

A *system configuration* is a tuple $C = (L, S, \tau, turn)$, where

- $L = \{L_i\}_{i \in ID}$ is a list of *agent configurations*; L_i is the configuration of the ith agent.
- $S : Var \mapsto Val$ is a *global context* mapping each shared variable to a corresponding valuation. $\mathbb{S}$ is the set of all possible global contexts.
- $\tau : \mathbb{R}^{\geq 0}$ is the *global time*.
- $turn \in \{\texttt{prog}, \texttt{env}\}$ is a binary bookkeeping variable; $turn = \texttt{env}$ indicates that all agents are done executing their programs for the current round, and the physical environment is evolving.

Update rules. The agent (and consequently, the global) configurations change according to *update rules*. The update rules are rewriting rules that model the changes in the agent configurations due to execution of agent programs as well as environment transitions. Agent programs consist of events and events consist of sequence of *Koord* statements. Thus, in describing the transition rules, we start with statement level transitions.

3.4 Statement semantics

Execution of a statement in a *Koord* program running on an agent, leads to the agent's configuration being updated. The statement-level updates are relations of the form:

$$\rightarrow_{Stmt} \subseteq (\mathbb{S} \times \mathbb{L} \times Stmts \cup \{\cdot\}) \times \mathcal{P}(\mathbb{S} \times \mathbb{L} \times Stmts \cup \{\cdot\}),$$

where *Stmts* refers to the set of all possible statements allowed by *Koord* syntax. The symbol '·' indicates an "empty" statement, which does not affect the configurations. That is, the update relation takes as input a tuple of (1) a global context, (2) an agent configuration, and (3) a statement, and maps it to a set of such tuples. As example, we give the semantic relations for some of the salient statement types in *Koord*.

3.4.1 Program variable updates. Rule LVAR-ASSIGN describes the semantics of how an assignment for a local variable x updates the agent configuration L to L' by modifying M. Specifically, $x \notin Keys(S) \wedge x \in Keys(M)$ represents the fact that there is no mapping from x in the global context S, but there is one in the local context M. The notation $pc = x = c \rightsquigarrow pc'$ represents that syntactic production pc' is the one following the statement $x = c$ from the production pc. $M[x \mapsto v]$ refers to the fact that in the map M, the key x maps to the value v.

$$\frac{\begin{array}{l} x \notin Keys(S) \wedge x \in Keys(M) \wedge pc = (x = c) \rightsquigarrow pc' \\ \wedge L = (pc, M, sm, \texttt{prog}) \wedge L' = (pc', M[x \mapsto c], sm, \texttt{prog}) \end{array}}{\langle S, L, x = c \rangle \rightarrow_{Stmt} \langle S, L', \cdot \rangle} \text{ LVAR-ASSIGN}$$

3.4.2 Shared variable updates. Similar to the rule LVAR-ASSIGN, rule SVAR-ASSIGN describes how the semantics of assignment to a shared variable x updates the current configuration. When an agent writes to a shared variable, it updates both its local copy in the local context M, as well as the global context S. The implementation of the runtime system for *Koord* message passing to send the updates to local variables to other agents.

$$\frac{\begin{array}{l} x \in Keys(S) \wedge L = (pc, M, sm, \texttt{prog}) \wedge S' = S[x \mapsto v] \\ \wedge L' = (pc', M[x \mapsto v], sm, \texttt{prog}) \wedge pc = (x = c) \rightsquigarrow pc' \end{array}}{\langle S, L, x = v \rangle \rightarrow_{Stmt} \langle S', L', \cdot \rangle} \text{ SVAR-ASSIGN}$$

3.5 Event semantics

The statement processing rules in Section 3.4 are applicable only when the turn of the agent is set to `prog`. As a result, statements in a program can only be executed during the program transition. We first present some rules for control flow in the program. The rule STMT-SEQ-1 ensures that a statement are processed completely before moving on to processing the next statement, and STMT-SEQ-2 ensures that once a statement that has been processed completely, the next statement comes up for processing.

$$\frac{\langle S, L, St_1 \rangle \rightarrow_{Stmt} \langle S', L', St_1' \rangle}{\langle S, L, St_1\ St_2 \rangle \rightarrow_{Stmt} \langle S', L', St_1'\ St_2 \rangle} \text{ STMT-SEQ-1}$$

$$\frac{}{\langle S, L, \cdot\ St_2 \rangle \rightarrow_{Stmt} \langle S, L, St_2 \rangle} \text{ STMT-SEQ-2}$$

$$\frac{\begin{array}{l} ev = \texttt{pre}\ Cond\ \texttt{eff}\ Ss \wedge pc' = pc \rightsquigarrow Ss \\ \wedge L = (pc, M, sm, \texttt{prog}) \wedge L' = (pc', M, sm, \texttt{prog}) \wedge eval(\texttt{pre}, L.M) \end{array}}{\langle S, L, \cdot \rangle \rightarrow_{Stmt} \langle S, L', Ss; endEvent \rangle} \text{ EVENT}$$

Given the control flow statements, rule EVENT states that only the effect of an an enabled event may be executed. The predicate $eval(pre, M)$ evaluates to `True` when the precondition *pre* of an event evaluates to *True* in the local context M of a given state. A marker(statement) *endEvent* is added after the effect of that event to indicate that the event has executed.

$$\frac{\begin{array}{l} L = (pc, M, sm, \texttt{prog}) \wedge \\ L' = (pc', M, sm, \texttt{env}) \wedge pc = endEvent \rightsquigarrow pc' \end{array}}{\langle S, L, endEvent \rangle \rightarrow_{Stmt} \langle S, L', \cdot \rangle} \text{ PROG-TO-ENV}$$

Rule PROG-TO-ENV states that after the event is executed, the *turn* of the agent is set to `env` indicating that after this execution, an environment transition occurs. The semantics doesn't specify an order of execution for the program transition (in other words, execution of an enabled event) by the agents in the system. We will demonstrate in Section 5 that the system can display non deterministic behaviors which arises from the agents in a system executing their events in different orders.

3.6 Alternating synchronous cyberphysical semantics

The advancement of time and environment transitions govern the changes in the system configuration. The system level program

transition rewrite rule is a mapping from an initial system configuration to a set of configurations. It has the type

$$\rightarrow_G \subseteq (\mathbb{L} \times \mathbb{S} \times \mathbb{R}^+ \times \{\texttt{prog}, \texttt{env}\}) \times \mathcal{P}(\mathbb{L} \times \mathbb{S} \times \mathbb{R}^+ \times \{\texttt{prog}, \texttt{env}\})$$

First, we present the semantics of executing the events for all agents. A *Koord* program can be seen as sequences of $\rightarrow_{Stmt}$ rules, which determine event executions followed by an environment transition. To express this, we define a rule that shows the the changes in the configuration of the overall system due to all the agents executing their program just before an environment transition is applicable.

$$\forall i \in ID \ \frac{(S, L_i, L_i.p) \rightarrow_{Stmt} (S', L'_i, p') \rightarrow_{Stmt} \ldots \rightarrow_{Stmt} (S'', L''_i, \cdot) \wedge L''_i.turn = \texttt{env}}{(L, S, \tau, \texttt{prog}) \rightarrow_G (L'', S'', \tau, \texttt{env})} \ \textsc{run}$$

The rule RUN expresses that starting from a global configuration $c = (L, S, \tau, \texttt{env})$, each agent i with local configuration L_i processes its program p using a sequence of $\rightarrow_{Stmt}$ rewrites, until its event is executed, and its *turn* set to env at the end of the event execution. Overall, the system goes from a configuration c to $c'' = (L'', S'', \tau, \texttt{env})$, with possibly different agent configurations and global context depending on whether any of the events executed resulted in writes to shared variables.

Now that we have the semantics for system to finish the program transition, we present the semantics for environment transitions, and how the *turn* resets to prog. Suppose the function $f : Var \times Sens \times Val \times \mathbb{R} \mapsto Val$ captures the behavior of the controller, and can be used to update the sensor ports after time δ. The rule ENVTRANS shows the semantics of the system configuration after the rule 3.6.

$$\frac{\forall i \in ID, L'_i.sm = f(L_i.M, L_i.sm, \delta) \wedge \forall x \in Keys(S), L'_i.M[x \mapsto S[x]] \wedge L_i.turn = \texttt{env}}{(L, S, \tau, \texttt{prog}) \rightarrow_G (L', S, \tau + \delta, \texttt{env})} \ \textsc{envtrans}$$

$$\frac{\forall i \in ID, L_i.turn = \texttt{env} \wedge L'_i.turn = \texttt{prog}}{(L, S, \tau, \texttt{env}) \rightarrow_G (L', S, \tau, \texttt{env})} \ \textsc{agent-env-to-prog}$$

$$\frac{\forall i \in ID, L_i.turn = \texttt{prog}}{(L, S, \tau, \texttt{env}) \rightarrow_G (L, S, \tau, \texttt{prog})} \ \textsc{env-to-prog}$$

Rule AGNT-ENV-TO-PROG is the semantic rule for setting the *turn* of each agent back to prog from env after the environment transition has been completed. The semantics synchronizes the program transitions of each agent, ensuring that an event execution for each agent happens every δ time.

Rule ENVTRANS captures the fact that the global context S is copied into local context of each agent i ($L_i.M$), thus ensuring that all agents have consistent shared variable values before the next program transition. After that, rule ENV-TO-PROG changes the turn of each agent back to program. In an actual execution, the controller would run the program on hardware, whose sensor ports evolve for δ time between program transitions.

3.7 Implementation in K

The language syntax is first parsed using a standard indentation parser implemented in python. We implemented *Koord* semantics in $\mathbb{K}$, which a rewriting-based executable framework for defining language semantics. One can view a language semantics naturally as a set of reduction rules over configurations. Components of a configuration are called *cells* in $\mathbb{K}$. To implement the semantics discussed in Section 3, the configuration in $\mathbb{K}$ includes several bookkeeping variables as well.

Semantics in $\mathbb{K}$ is expressed using configurations, which organize the components in elements called *cells*. Cells are labelled, have types indicating what kind of elements can be contained in them, and help specify the rewrite rule in context. $\mathbb{K}$ allows underspecification of rewrite rules, meaning, only the rewrite rules affecting part of the configurations need to be specified if the rule doesn't affect the other parts of the rule. Our notion of configurations translate in a straightforward manner to $\mathbb{K}$ configurations. We present a couple of example rewrite rules in $\mathbb{K}$ to demonstrate how the executable semantics is implemented.

We use a top level *system* cell with nested cells corresponding to the elements discussed in Section 3, as well as other cells which are used to store information like number of agents in the system, simulation parameters of the program, and other bookkeeping information we used to implement the semantics. There is a special cell called the *k* cell, which is used to store the current computation in the program. Each agent has a *k* cell to store its computation. Recall that a local variable is updated only in the local memory. The rule for this checks whether the variable is not in the keys of the map *sysLoc* from shared variables to their locations. The following rule in $\mathbb{K}$ that corresponds to our LVAR-ASSIGN rule in the semantics.

```
<agent> ... <k> V:Id = I:Val => .  ... </k>
            <loc> ... V |-> L ... </loc>
            <locstore> ... L |-> _ => I ... </locstore>
         ... </agent>
<sysloc> Rho </sysloc>                    when L notBool inKeys(Rho)
```

We use bookkeeping variables in the $\mathbb{K}$ configuration to ensure that all agents have their turn set to env before the system environment transition.We also ensure that an agent has updated its sensor ports after the environment transition, to implement the rule AGENT-ENV-TO-PROG. Once all the agents in the system are ready to perform the next program transition, the rule ENV-TO-PROG is implemented as follows. If the number of agents which have set their *turn* to *env* (indicated by the cell *envToProg*) is the same as the number of all agents in the system, then the system sets its turn from *Env* to *Prog*.

```
<system>...
 <envToProg> N </envToProg>
 <numAgents> N </numAgents>
 <turn> Env => Prog </turn> ...
</system>
```

4 *Koord* SOFTWARE STACK

4.1 Runtime system

The *Koord* runtime system comprises of a program which interacts with the controller plane through sensor and actuator ports, which in turn communicates with the physical plant through ROS messages. The software stack implementing this uses the *Koord* front-end, that takes as input a *Koord* program and generates a Python program, that runs on a middle layer (written in Python) on the agents in the system (either in simulation or on actual hardware). In actual deployment, Python middle layer communicates through ROS messages with the ROS interfaces for the low level hardware controls.

The Python middle layer in our implementation of the *Koord* language consists of the following major modules:(1) a communication module implementing the shared memory semantics, (2) a motion module which in simulation implementing the dynamics of the environment, and in hardware, is responsible for publishing and subscribing to ROS messages between the program and the platform. (3) an agent thread module which implements the discrete event loop semantics of each agent. .

To run a *Koord* program (hardware or simulation), the user has to provide a configuration file, with (1) the number of agents, (2) in case of simulation, the initial positions of the agents and the length of the simulation and (3) in case of hardware deployment their IP addresses, and the localization system. Each agent in the system launches the agent thread, which has its own *Motion automaton* thread, which implements the controller for the agent thread. Each agent thread also has a communication thread, which it uses to listen for and send messages for shared variable updates to other agent threads. In simulation, the *Koord* program launches N_{sys} threads, each with its own communication and motion automaton threads, and runs the execution as long as specified in the program.

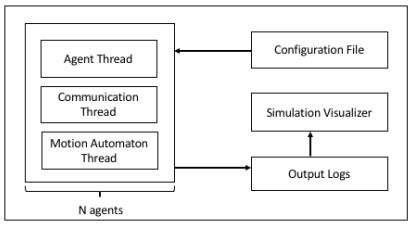

Figure 5: Simulation of a *Koord* program

Addressing the gap between theory and implementation. Usually cyberphysical system models are based on mathematical representations of the system without an emphasis on the implementability of said model. Our formal *executable* semantics is intended to narrow that the gap between the theory and implementation of cyberphysical systems, as the translation of the mathematical rules into exact rewrite rules in the $\mathbb{K}$ -semantic framework ensures that.

4.2 Key environment assumptions

In the designing the formal semantics of *Koord*, the following assumptions are made. Our eventual goal is to be able to provide safety guarantees for *Koord* applications under these assumptions and also ensure that hardware platforms can meet these assumptions.

Periodic event execution semantics. Our semantics assumes periodic execution of agent programs, with minimum period defined by the sampling parameter $\delta > 0$. This is a standard programming style in embedded and control systems [**?**]. A related assumption is that the program transitions in a round have zero logical execution time. To this end, our implementation runs barrier synchronization across agents in each round, and we ensure that agent controllers only starts driving the physical plant after the maximum possible time to execute a program transition step has elapsed.

Shared variable implementation over message passing. For the shared memory abstraction to be faithfully implemented on a message-passing network, the message delays between agents should at least be less than the the sampling parameter δ. We use TCP/UDP to send agent shared variable update messages over wifi in our implementation. The messages are propagated during the *environment transition* in each round, and processed before the next round of program transitions starts.

Known set of participants. The number and identities of the participating agents *ID* is known to each agent. In distributed systems with failures, computing the set of participants is a well-studied and foundational problem. problem [**?**] and it is related to the active area of research on failure detectors [**? ?**]. In the future, one could relax this assumption. For example, in our current hardware deployment, we are using the *ID* as a sensor port that is informed of the identity of the live processes by periodic messages.

4.3 Verification with Bounded Model Checking

We built up on the capabilities of the $\mathbb{K}$ semantic framework to provide a verification tool on top of the executable semantics of *Koord*, which checks bounded invariants (n-invariants) for *Koord* applications using explicit-state reachability analysis. The input to the verification tool is (i) P: the program, (ii) *inv*: a candidate invariant function, (iii) ID: set of agents, (iv) δ: time step size and (v) n: number of rounds of the execution to check.

The tool outputs 'safe' if the *inv* is indeed an n-invariant, 'unsafe' if it finds a counter-example execution of length n. In some cases, it can return 'unknown' as the reachable set computed by the verification algorithm are an overapproximation of the actual reachable set of configurations.

The verification algorithm uses the *Koord* executable semantics to produce all configurations that the system reaches during program transitions. For this purpose, our semantics allows values of variables to be stored as intervals and propagated through program execution and environment transitions on explicit system models. If the dynamics of the system is a black box model then sensitivity analysis tools can be used to complement the reachable state computation. The actual verification algorithms are beyond the scope of this paper, and will be discussed in future submissions.

Benchmarks. The applications whose verification results are shown in Table 1 are the following:

(i) a simple HVAC model that should ensure that the temperature of several rooms should remain above a given temperature, (ii) the above *Task* app, that should ensure that there are no collisions, (iii) Fischer's mutual exclusion protocol, (iv) and a Platooning system, that should ensure that a platoon of cars maintains a separation between any consecutive cars.

The *Agents* column in the table refers to the number of agents in the system. The purpose of our verification module is to support a semantics driven design of reliable and safe distributed system i.e. assumptions that were required to make correctness guarantees about a program with well-defined *safe* behaviors were used to make design decisions about the language semantics.

The state space of a distributed system of agents with this level of non-determinism increases exponentially in size with the number of agents and the number of rounds. Although, this verification

Table 1: Verification Summary

Benchmark	Agents	Num. of Rounds	Time to Verify	Safe
HVAC	3	10	2m18s	Safe
HVAC	4	10	2m51s	Safe
HVAC	3	10	38s	Unsafe
HVAC	4	10	43s	Unsafe
Task	2	10	1m8s	Safe
Task	3	10	3m6s	Safe
Task	3	10	4m13s	Unsafe
Task	4	10	8m29s	Unsafe
Fischer's	2	10	3m4s	Safe
Fischer's	2	10	1m19s	Unsafe
Platooning	3	10	1m58s	Safe
Platooning	4	10	3m29s	Safe
Platooning	3	10	1m28s	Unsafe
Platooning	4	10	1m43s	Unsafe

method currently only scales to small problem instances, in context,this is a promising tool connecting a high-level language to actual implementations.

5 THREE *Koord* APPLICATIONS

The *Koord* simulator provides the user a way to test and debug application programs with different and heterogeneous physics models. We present three case studies to demonstrate some of the key features of the *Koord* language and its simulator.

5.1 Platooning

Figure 6 shows the *Koord* program for a simplified platooning protocol for vehicles on a single lane. The position of successive vehicles is communicated through **allread** shared variable x. The goal is to maintain a minimal *safe* separation of 5 meters between any two consecutive cars. For the sake of simplicity, here the leading car has a preprogrammed behavior (it sets its acceleration to 2 or -2 every 100 rounds); in a more realistic deployment this behavior could be based on speed limits or other traffic. Every other car in the platoon with pid i sets the actuator, namely, its acceleration *Motion.xacc* based on its distance from the car with pid $i+1$; slowing down when the separation is less than the *threshold* of 8 meters, accelerating otherwise.

```
Platoon
using Motion
  sensors pos psn
  actuators float xacc

allread:
  pos x = Motion.psn

local:
  float threshold = 8.0
  float dist
  int timer = 0
  int mode = 0

SetAcc:
 pre True
 eff
   if (pid == N_sys − 1):
     timer = timer + 1
     if (timer == 100):
       timer = 0.0
       mode = (mode + 1) mod 2
       Motion.xacc = 2.0 − mode ∗ 4
   else:
     dist = pos[pid + 1].x − pos[pid].x
     if (dist >= threshold):
       Motion.xacc = 1.0
     else:
       Motion.xacc = −1.0
   x[pid] = Motion.psn
```

Figure 6: *Koord* program for robot i in a platoon to adjust its x acceleration accordingly .

Recall that the synchronous semantics of *Koord* implies that in each round the participating vehicles update their modes and accelerations based on the positions of others updated from the last round. These updates can happen in arbitrary order but all in zero logical execution time. The resulting decisions affect the actuation (*Motion.xacc*) for a period of δ time, before the execution of the next round.

Figure 7 shows the simulator generated position x vs *time* plots for different values of δ, for four vehicles.

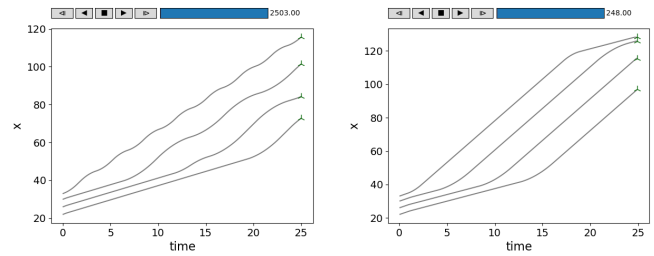

Figure 7: Platoon of four cars. *Left* for $\delta = 0.01$, the cars eventually get to safe separations, while reacting to the first car changing its acceleration periodically and *Right* for $\delta = 0.1$, this shows an unsafe execution where the simulation stops upon a collision.

A *correct* program should ensure that at no point, the separation along the x-axis between two of agents is less than the required safe distance (8 m). Note, that the variable *threshold* needs to be set to be higher for larger values of the round duration δ. If δ is too long, then the cars will not slow down even when the separation between then might have become less than the *threshold*. We verified using the *KoordBMC* tool that *Platoon* is safe for four cars when δ is small enough and has potential unsafe behaviors otherwise. Figure 7(Right) shows the x vs *time* for a counterexample with δ is too long. For a threshhold value of 8.0, required minimum separation of 5.0, for $\delta = 0.01$ *KoordBMC* showed that the program was safe for 10 rounds starting at an initial configuration where each car i is at a distance of 6.0 from the car $i+1$, For $\delta = 0.1$ *KoordBMC* showed that the program was unsafe.

5.2 Swarm formation

We used *Koord* to implement several standard pattern formation protocols like the *Lineform* app from the Introduction. The shared *allread* variable x is used by the agents to communicate their position to the other robots. The function *mid* computes the centroid of a list of vectors.

With the addition of an explicit neighbor relation, the *Squareform* app as shown in Figure 8 can be used to generate a rhombus shape in the plane formed by four extremal points. The code defines a local variable *nbrs* which is a static list of list of integers, where the ith element is a list of the *pid*s of the neighbors of the agent with *pid* i. The list of *pid*s of the extremal points is stored in a list of integers called *corners*. If the agent *pid* is not in *corners*, then it moves to the mid point of all its neighbors. [4]

[4]We omit the actual initialization of the variables *nbrs* and *corner*, as it depends on the simulation parameters.

```
1  Squareform
2   using Motion:
3    ...
4   allread:
5    pos x
6
7   local:
8    List<List<int>> nbrs = ...
9    List<int> corners = ...
11 TargetUpdate:
12  pre True
13  eff if not(pid in corners):
14    Motion.target = mid(x in nbrs)
15    x[pid] = Motion.psn
```

Figure 8: ***Koord*** **application forming a shape in 3D defined by the static corners.**

Figure 2 shows two screenshots of the 3d-simulation for the *Shapeform* app with 25 agents. Here the *Motion* controller used is a simple drone model. Instead of the *mid* function, other operators on the positions of the neighboring agents could be applied to create different swarming protocols. In addition, using the scripting technique for the leading vehicle in *Platoon*, the swarming robots can be guided to move in formation.

5.3 Distributed task allocation

Figure 4 shows the distributed task allocation application written in *Koord*. Distributed task allocation is challenging as it involves mutual exclusion as well as construction of de-conflicted paths to task locations. Tasks can be abstractions for real location-based objectives like package delivery, surveillance, or fire-fighting.

We implement a task allocation strategy using shared variables for coordination. Mutual exclusion is always an essential feature when shared variables are involved. *Koord* provides a locking mechanism using the keyword *atomic*, which the *Assign* event uses to update the shared list of tasks *taskList* safely.

Here the motion module implements Dubin-like vehicle models for the agents, and RRT-based path planning to avoid static obstacles specified in map. Figure 9 shows two screenshots of a *Task* application simulations.

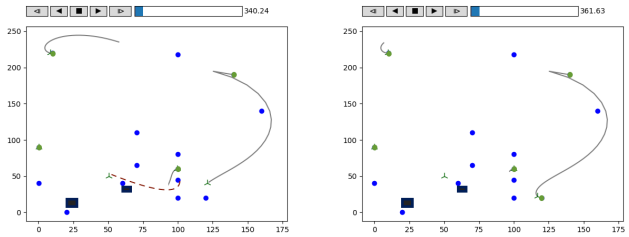

Figure 9: The grey trails represent the positions of the five agents in the last 150 **time units; task locations of the incomplete tasks (blue), completed tasks (green), static obstacles (black), currently computed infeasible path (dashed red).**

We could easily replace the *Motion* automaton (consequently, the dynamical model) of any of the agents, and run the same code. Using different control parameters, and path planning parameters for the same motion automaton we obtain different statistics for task completion and distance traversal as depicted in in Figure 10.

This heterogeneity and portability can be useful for coordinating the same application between different hardware platforms. We ran the task application for various configurations of cars (moving in 2d) and quadcopters (moving in 3d) for the task application, and present some completion statistics below. The agents performed 25 tasks, with three obstacles. An agent (car or quadcopter) is considered to have reached a task location if the x and y coordinates of the task are within a specified minimum distance of the current x and y coordinates of the agent.

For all agents operating with the same motion automaton and dynamics model, we observed that conflict resolution took more time with more agents, as expected (Figure 10). The completion time remained relatively stable. The maximum and minimum distances travelled were relatively closer with more robots. However, all these results were affected by the non-determinism in choice of paths and tasks by the agents, and their previous relative positions at the moment of choosing the next task as well.

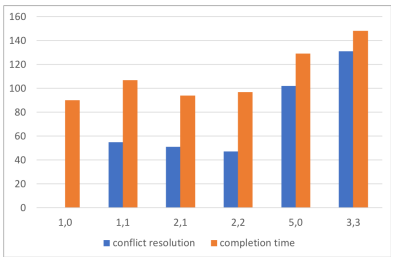

Figure 10: Completion time (orange) and conflict resolution time (blue) for different agent configurations (quadcopters, cars).

These results demonstrate a capability of covering various scenarios with different levels of heterogeneity and coordination.

5.4 Conclusion

The divide between theory and implementation concerns of cyberphysical systems is an obstacle in development of portable, reusable and safe applications. Programming language research in this area can present a unified approach to bridging the aforementioned divide. We have presented an event-triggered language for coordination and control, and implemented its formal executable semantics. The shared memory based communication model we used enables us to naturally code protocols for robotics and distributed systems. Our work is a move in the direction of providing users without programming expertise with an ability to safely program distributed applications. The *Koord* language uses abstractions for components like path planning, localization, and shared memory, to simplify code and separate concerns. We also discussed how varying relevant system parameters affect their analysis.

The *Koord* semantics model is realistic by design to deploy on hardware. Our shared memory semantics are informed by the actual implementation using message passing protocols, the language and platform interfaces are used to define the configuration elements on which rewrite rules are specified. Conversely, the formal analysis driven by the semantics guides the requirements upon the hardware as well, narrowing the gap between theory and implementation of the language semantics.

Koord and supporting compiler can enable users to develop and run distributed robotics applications in a platform independent

fashion; *Koord* programs can be ported across platforms automatically by linking with the right platform dependent libraries; and the *Simulator* and *KoordBMC* tools can be used to simulate and verify the same application code.

A KOORD SYNTAX

```
Program ::= Defs [Module+] DeclBlock [Init] Event+

Defs ::= [FuncDef+] [AdtDef+]
FuncDef ::= def fun ⟨identifier⟩ (Param+) : Stmt+
AdtDef ::= def adt ⟨identifier⟩ : Decl+
Param :: Type ⟨identifier⟩

Module ::= using module ⟨identifier⟩ : SPorts APorts
SPorts ::= sensors : Decl+
APorts ::= actuators : Decl+

Decl :: Type ⟨identifier⟩ | Type ⟨identifier⟩ = Val
Type ::= int | float | bool | pos | adt
       | Type[Int] | List ⟨Type⟩ | Queue ⟨Type⟩

DeclBlock ::= AWDecls ARDecls LocDecls
AWDecls ::= allwrite : Decl+
ARDecls ::= allread : Decl+
LocDecls ::= local : Decl+

Init ::= init : Stmt+
Event ::= ⟨identifier⟩ : pre [ urgent ] (Cond) ;
        eff : Stmt+

Expr ::= AExpr|BExpr
AExpr ::= AExpr AOp AExpr
        | Expr ++ | - AExpr | Var | AVal
AOp ::= + | − | ∗ | / | %
BExpr ::= Expr RelOp Expr|Expr COp Expr
        | not Expr | Var | BVal
RelOp ::=≥|≤|≥|==|>|<|≠
COp ::= and | or

Stmt ::= Assign | FnCall | Atomic
       | Ite | Loop | Return
Assign ::= Var = Expr
Ite ::= if BExpr Stmt+ [ else Stmt+ ]
FnCall ::= ⟨identifier⟩ ([AExpr | BExpr]+)
Atomic ::= atomic : Stmt+
Loop ::= For ⟨identifier⟩ in AExpr : Stmt+
Return ::= return Expr | return

Var ::= ⟨identifier⟩ | ⟨identifier⟩ [Expr]
      | ⟨identifier⟩ . ⟨identifier⟩
Val ::= AVal | BVal
AVal ::= Int | Float
BVal ::= Bool
```

Figure 11: *Koord* program syntax.